

SOFTWARE ARCHITECTURE & DESIGN SPECIFICATION

Blood Bank Management System (BBMS)

Item	Details
Project	Blood Bank Management System (BBMS)
Document	Software Architecture & Design Specification
Deliverable	Software Engineering Mini-Project — Part 1
Version	1.0
Date	27 September 2026
Team	Team T9
Authors	(PES1UG24CS697); (PES1UG24CS701); t (PES1UG24CS705);(PES1UG24CS709)
Technology	Python (Django) / JavaScript (Node.js), JavaScript/HTML/CSS, SQLite

Table of Contents

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 References
 - 1.4 Definitions and Acronyms
 - 1.5 Document Overview
- 2. Architectural Design
 - 2.1 Architecture Overview
 - 2.2 Architectural Goals
 - 2.3 Architectural Pattern
 - 2.4 Architectural Constraints and Assumptions
 - 2.5 Quality Attribute Drivers
 - 2.6 System Context
 - 2.7 Component Architecture
 - 2.8 Component Descriptions
 - 2.9 Component Interaction
 - 2.10 Data Architecture
 - 2.11 Deployment Architecture
- 3. Security Architecture
 - 3.1 Security Objectives
 - 3.2 Security Architecture Overview
 - 3.3 Authentication and Session Security
 - 3.4 Authorization and RBAC
 - 3.5 Input and Data Protection
 - 3.6 Rate Limiting and Abuse Protection
 - 3.7 Audit and Accountability
 - 3.8 Security Traceability
- 4. Detailed Design
 - 4.1 Design Principles
 - 4.2 Module Design
 - 4.3 Sequence Diagram — Donor Appointment
 - 4.4 Sequence Diagram — Hospital Requisition and Dispatch
 - 4.5 API Design
 - 4.6 API Authentication and Authorization
 - 4.7 Error Handling
 - 4.8 Validation and State Management
 - 4.9 Audit Logging Design
- 5. Requirements Traceability
 - 5.1 Architecture-to-SRS Traceability
 - 5.2 Design-to-Test-Plan Traceability
- 6. Design Decisions, Risks and Limitations
 - 6.1 Key Design Decisions
 - 6.2 Risks and Mitigations
 - 6.3 Limitations
- 7. Conclusion

1. Introduction

1.1 Purpose

This Software Architecture & Design Specification defines the structural and detailed design of the Blood Bank Management System (BBMS) described in the SRS. It translates the functional, non-functional, and security requirements into an implementable web application architecture, component decomposition, interfaces, interaction flows, API contract, error-handling approach, and security architecture. The specification is written for Team T9, course faculty/evaluators, and future maintainers of the academic prototype.

1.2 Scope

The design covers donor, hospital, blood bank staff, and administrator workflows; blood collection and unit identification; serology and quarantine; component separation; inventory and FEFO; requisitions and STAT handling; ABO/Rh compatibility and allocation; dispatch and delivery; audit logging; reporting; and public availability. The design does not introduce integrations or capabilities excluded by the SRS, including national registry integration, payment processing, direct laboratory equipment integration, native mobile applications, or multi-language localization.

1.3 References

- BBMS Software Requirements Specification v1.0, Sections 1–8, provided as Deliverable Part 1.
- BBMS Software Test Plan v1.0, including security validation, test cases, and traceability.
- IEEE Std 830-1998, used by the project SRS as a structural reference.

1.4 Definitions and Acronyms

Term	Meaning
BBMS	Blood Bank Management System
RBAC	Role-Based Access Control
FEFO	First-Expiry-First-Out
API	Application Programming Interface
REST	Representational State Transfer
ORM	Object-Relational Mapping
ABO/Rh	Blood group compatibility classification
STAT	Emergency / immediate-priority requisition
Bag ID	Unique identifier of a collected blood unit
SRS	Software Requirements Specification

1.5 Document Overview

Section 2 defines the architecture and its components. Section 3 defines the security architecture and maps controls to SRS security requirements. Section 4 describes detailed module design, sequence diagrams, APIs, validation, state management, and error handling. Section 5 provides traceability from architecture/design elements to the SRS and companion Test Plan. Section 6 records major design decisions, risks, and limitations.

2. Architectural Design

2.1 Architecture Overview

BBMS is designed as a centralized client-server web application with a browser-based frontend, a RESTful JSON API, modular application services, an ORM/data-access layer, and an embedded SQLite database. The architecture is intentionally modular so that donor, hospital, inventory, matching, dispatch, and audit responsibilities remain separated even though the application may be deployed as a single process for the mini-project.

2.2 Architectural Goals

- Separate presentation, API/control, business logic, data access, and persistence responsibilities.
- Enforce server-side authentication and RBAC for every protected function.
- Protect blood-unit safety state transitions and maintain auditability of data-changing actions.
- Support the measurable SRS quality requirements for performance, reliability, accessibility, portability, maintainability, and future scalability.
- Keep the design compatible with either the Django/Python or Node.js/Express backend option stated by the SRS.

2.3 Architectural Pattern

The architecture uses a layered client-server pattern with an MVC-inspired web/API implementation. The logical layers are: (1) Presentation/Web Client, (2) REST API / Controller layer, (3) Application Services, (4) Persistence / ORM, and (5) SQLite storage. Django or Express can implement the web/API layer, while the service and persistence boundaries remain conceptually stable. This separation directly supports BBMS-NF-010 and keeps changes localized by module.

2.4 Architectural Constraints and Assumptions

- SQLite is an embedded database and the SRS limits expected concurrency to the scale of a university mini-project.
- The server is authoritative for timestamps, expiry calculations, and audit events.
- Physical serology is performed outside BBMS; staff enter the resulting Pass/Fail values manually.
- SMTP is an external notification dependency used for optional email notifications.
- The application is a prototype and is not certified for clinical or regulatory use.

2.5 Quality Attribute Drivers

Quality Attribute	Design Response	SRS References
Performance	Indexed queries, lightweight REST endpoints, ORM access, FEFO query ordering	NF-001–NF-003
Reliability	Transactional persistence, restart-safe committed data, controlled status transitions	NF-004–NF-006
Usability / Accessibility	Role-specific dashboards, field-level validation, WCAG 2.1 AA target	NF-007–NF-009
Maintainability	Layered modules and service boundaries; automated test target	NF-010–NF-011
Portability	Browser-based client; backend isolated behind framework-neutral interfaces	NF-012–NF-013
Scalability	Location-aware inventory model and data-access abstraction	NF-014–NF-015
Security	Authentication, RBAC, validation, secure cookies, rate limits, audit logging	SR-001–SR-008

2.6 System Context

External users interact with BBMS through a modern web browser. The frontend exchanges JSON with the REST API over HTTPS. The API delegates authenticated work to domain services and persists data through an ORM or equivalent data-access layer into SQLite. Notifications are sent through SMTP. The public availability page uses only sanitized stock data and does not expose donor- or hospital-identifying information.

2.7 Component Architecture

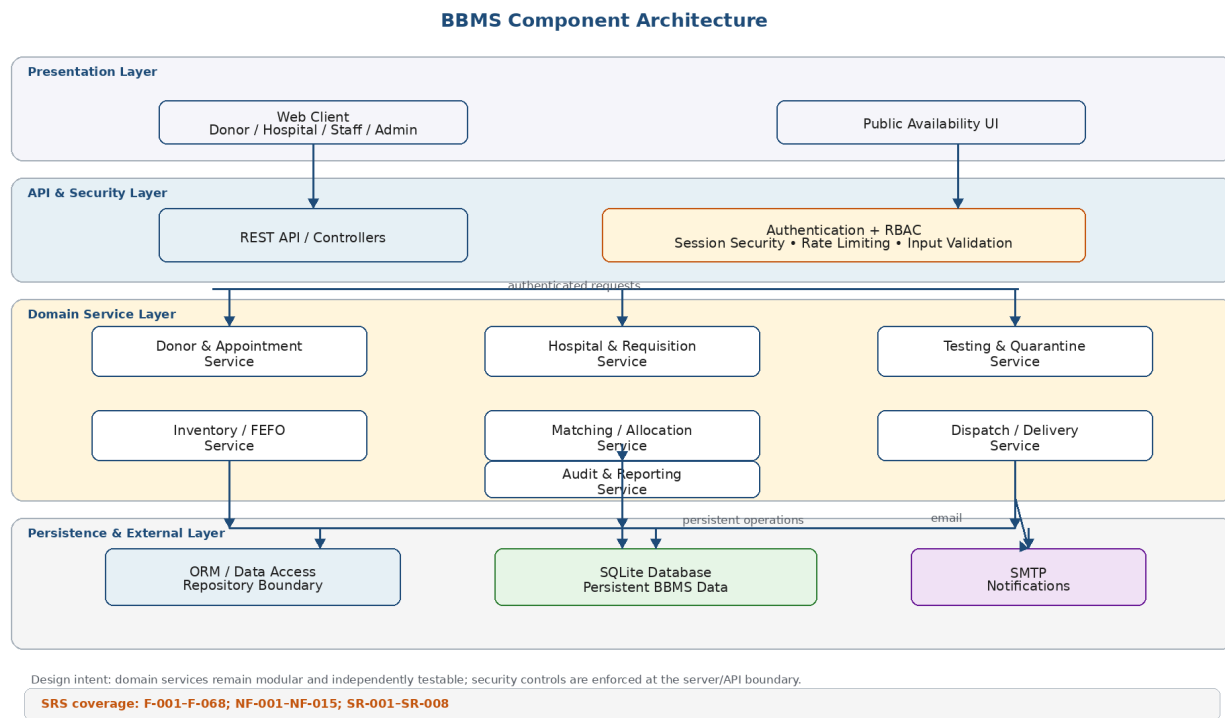


Figure 2.1 — BBMS Component Architecture

The component diagram shows the main application responsibilities and their key interactions. Authentication and RBAC are cross-cutting controls rather than business-domain modules; Audit & Reporting is similarly cross-cutting for accountability and operational reporting.

2.8 Component Descriptions

Component	Responsibility	Primary SRS Coverage
Web Client	Role-specific dashboards, forms, validation feedback, public availability UI	F-001–F-068; NF-007–NF-009
REST API Layer	HTTP routing, request/response mapping, validation entry points, controller logic	F-001–F-068; SR-001–SR-008
Authentication & RBAC	Credential verification, session lifecycle, role authorization, logout/rate limiting	F-001–F-005; F-040–F-042; F-060–F-063; SR-001–SR-007

Donor Management	Profiles, eligibility and appointment workflows	F-006–F-019
Hospital & Requisition	Hospital onboarding, requisitions, STAT priority, tracking	F-040–F-048
Testing & Quarantine	Serology results, quarantine/discard rules, component separation	F-024–F-032
Inventory / FEFO	Units, counts, status, expiry, FEFO selection, location inventory	F-020–F-039
Compatibility & Allocation	ABO/Rh matching, cross-match outcomes, reservations	F-049–F-052
Dispatch & Delivery	Dispatch records, custody confirmation, status transitions	F-053–F-057
Audit & Reporting	Immutable/tamper-evident events, operational reports, CSV export	F-058–F-066; SR-008
Public Availability	Unauthenticated, privacy-preserving stock view	F-067–F-068
SQLite via ORM	Persistent storage and transaction boundary	NF-004–NF-006; SR-004

2.9 Component Interaction

- Donor booking calls Eligibility before Appointment Service creates the appointment.
- Blood collection creates a Blood Unit in Quarantined state and updates donor history.
- Serology/Quarantine controls transition units to Available or Discarded and feeds Inventory.
- Hospital requests are validated, prioritized, and passed to Matching/Allocation for compatible FEFO candidates.
- Dispatch records change allocated units to Dispatched; hospital receipt changes them to Delivered.
- All data-changing and security-relevant actions emit audit events.

2.10 Data Architecture

The logical data model is derived from the SRS domain objects. Core entities include User/Role, Donor, Hospital, Appointment, Donation/Collection, BloodUnit, SerologyResult, BloodComponent, Inventory/Location, Requisition, Allocation/CrossMatch, Dispatch, DeliveryConfirmation, AuditLog, and reportable aggregates. Foreign-key relationships preserve ownership and traceability. The design keeps separated component records linked to their originating Bag ID, and keeps requisition-to-allocation-to-dispatch history intact.

2.11 Deployment Architecture

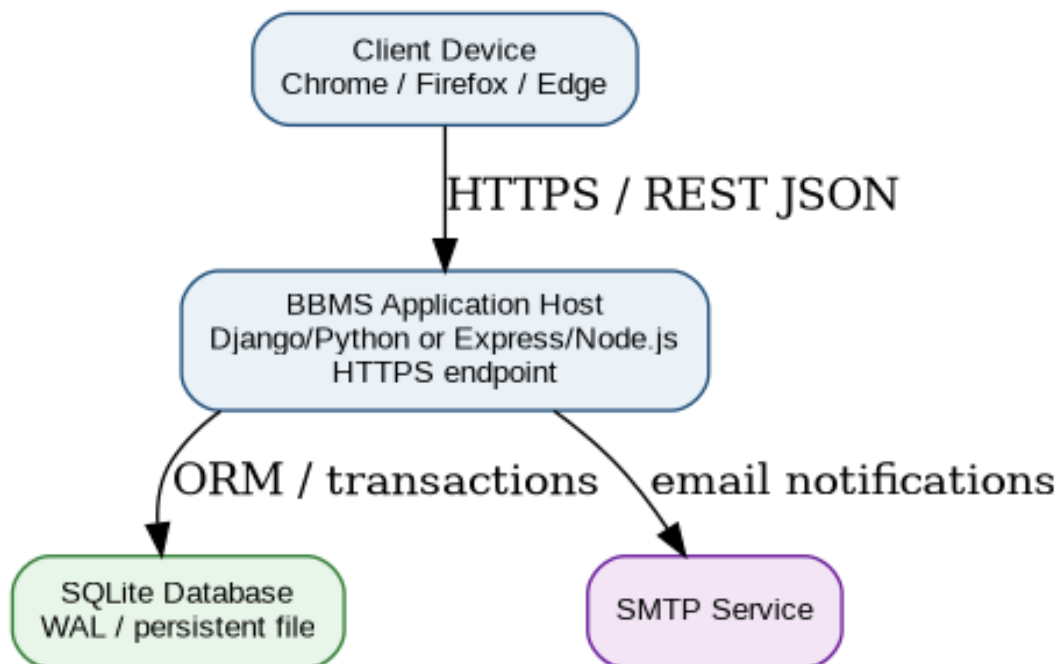


Figure 2.2 — Logical Deployment Architecture

3. Security Architecture

3.1 Security Objectives

- Protect donor personal and medical information from unauthorized access or disclosure.
- Ensure only authenticated and authorized users perform role-appropriate operations.
- Preserve blood-unit testing and status integrity so unsafe blood cannot be allocated or dispatched.
- Maintain accountability through security and data-change audit records.

3.2 Security Architecture Overview

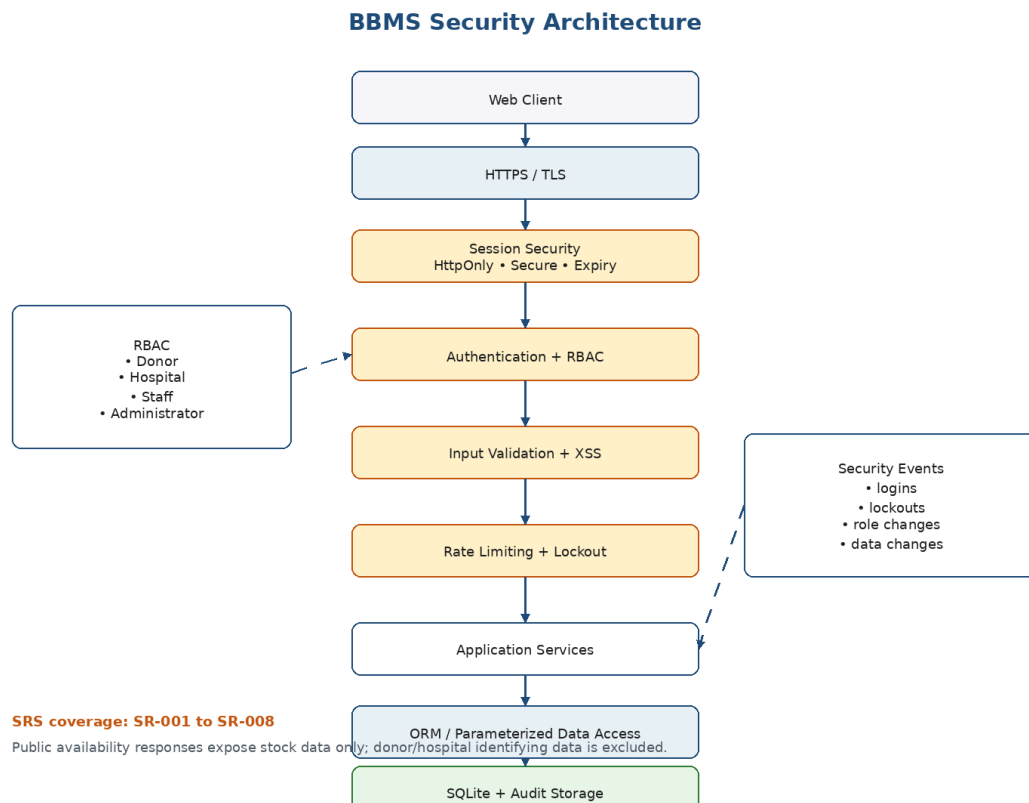


Figure 3.1 — BBMS Security Architecture

Security is enforced at multiple layers: encrypted transport, session protection, server-side authentication and RBAC, input validation and output encoding, abuse controls, ORM/parameterized data access, and audit logging. Client-side controls are treated as usability aids only; authorization decisions are made on the server.

3.3 Authentication and Session Security

- Require every role to authenticate before accessing role-specific functions (SR-001).
- Store passwords as salted one-way hashes; never log or return plaintext credentials (SR-002).
- Issue session tokens using HttpOnly and Secure cookies with a defined expiry; invalidate on logout and after 30 minutes of inactivity (SR-006).
- Use HTTPS/TLS for client-server communication (SR-008).

3.4 Authorization and RBAC

Every protected API endpoint is protected by server-side role checks. A typical authorization context includes user identity, role, and account state. Donor accounts cannot access staff/admin functions; unapproved hospitals cannot submit requisitions; administrators can access account and audit management features. Unauthorized protected requests return HTTP 403, as required by SR-003.

3.5 Input and Data Protection

- Validate data types, ranges, required fields, and business constraints before service execution.
- Use ORM queries or parameterized statements to avoid constructing SQL from raw user input (SR-004).
- Sanitize/validate submitted input and encode browser-rendered output to prevent reflected and stored XSS (SR-005).
- Exclude donor-identifying and hospital-identifying data from the public availability response (F-068).

3.6 Rate Limiting and Abuse Protection

Login requests are limited to no more than 10 attempts per minute per source IP. An account is locked for 15 minutes after five consecutive failed attempts. The rate limiter and lockout counter are applied before expensive authentication work where possible and are logged as security events, satisfying SR-007 and the audit portion of SR-008.

3.7 Audit and Accountability

Audit events are generated for creations, status changes, and deletions of donors, blood units, requisitions, and user accounts, including actor, action, timestamp, and affected record ID. Security events such as login attempts, lockouts, and role changes are

also recorded. The full audit log is restricted to administrators and rendered read-only/non-deletable in the application, supporting F-058, F-059, and SR-008.

3.8 Security Traceability

SRS Security Requirement	Architecture Control	Test Plan Coverage
BBMS-SR-001	Authentication service and protected API middleware	TC-SEC-01
BBMS-SR-002	Password hashing in authentication service; credential secrecy	TC-SEC-02
BBMS-SR-003	Server-side RBAC middleware; HTTP 403 on unauthorized access	TC-SEC-03
BBMS-SR-004	ORM / parameterized data-access layer	TC-SEC-04
BBMS-SR-005	Schema validation + output encoding / XSS defenses	TC-SEC-05
BBMS-SR-006	HttpOnly/Secure cookies, expiry, logout, inactivity timeout	TC-SEC-06
BBMS-SR-007	Login logout + per-IP rate limiting	TC-SEC-07
BBMS-SR-008	HTTPS/TLS + security-event / data-change audit trail	TC-SEC-08, TC-AUD-01

4. Detailed Design

4.1 Design Principles

- Single responsibility: each business module owns one cohesive domain responsibility.
- Explicit state transitions: unsafe or expired units are never treated as available by default.
- Server-side validation and authorization are authoritative.
- Traceability first: records retain IDs and timestamps needed to follow donor → collection → unit → requisition → dispatch → delivery.
- Stable API contracts: request/response structures are defined independently of the chosen backend framework.

4.2 Module Design

Module	Key Operations	Important Rules / SRS
Donor	Register, authenticate, profile, questionnaire, appointment	18+ registration; unique email; eligibility before booking; 90-day default interval; F-001–F-016
Collection / Blood Unit	Record collection, volume/time, create Bag ID, blood group	Unique Bag ID; lab-confirmed blood group distinct from self-report; F-017–F-023
Serology / Quarantine	Record Pass/Fail, status transitions, discard	Default Quarantined; all required tests Pass before Available; any Fail → Discarded; F-024–F-030
Components / Inventory	Separate units, inventory counts, expiry, FEFO	Trace components to source Bag ID; exclude unsafe/expired; F-031–F-039
Hospital / Requisition	Register, approve hospital, create requests, STAT	Unapproved hospitals cannot submit; unique Pending request; F-040–F-048
Matching / Allocation	Compatibility list, cross-match, reserve units	Compatible groups only unless explicit authorized override; F-049–F-052
Dispatch / Delivery	Create dispatch, update unit/request, confirm receipt	Record staff/timestamp; Dispatched then Delivered; F-053–F-057
Audit / Reports	Audit records, admin view, CSV reports	Read-only audit display; activity and inventory reports; F-058–F-066
Public Availability	Show stock summary	Unauthenticated; no identifying data; F-067–F-068

4.3 Sequence Diagram — Donor Appointment

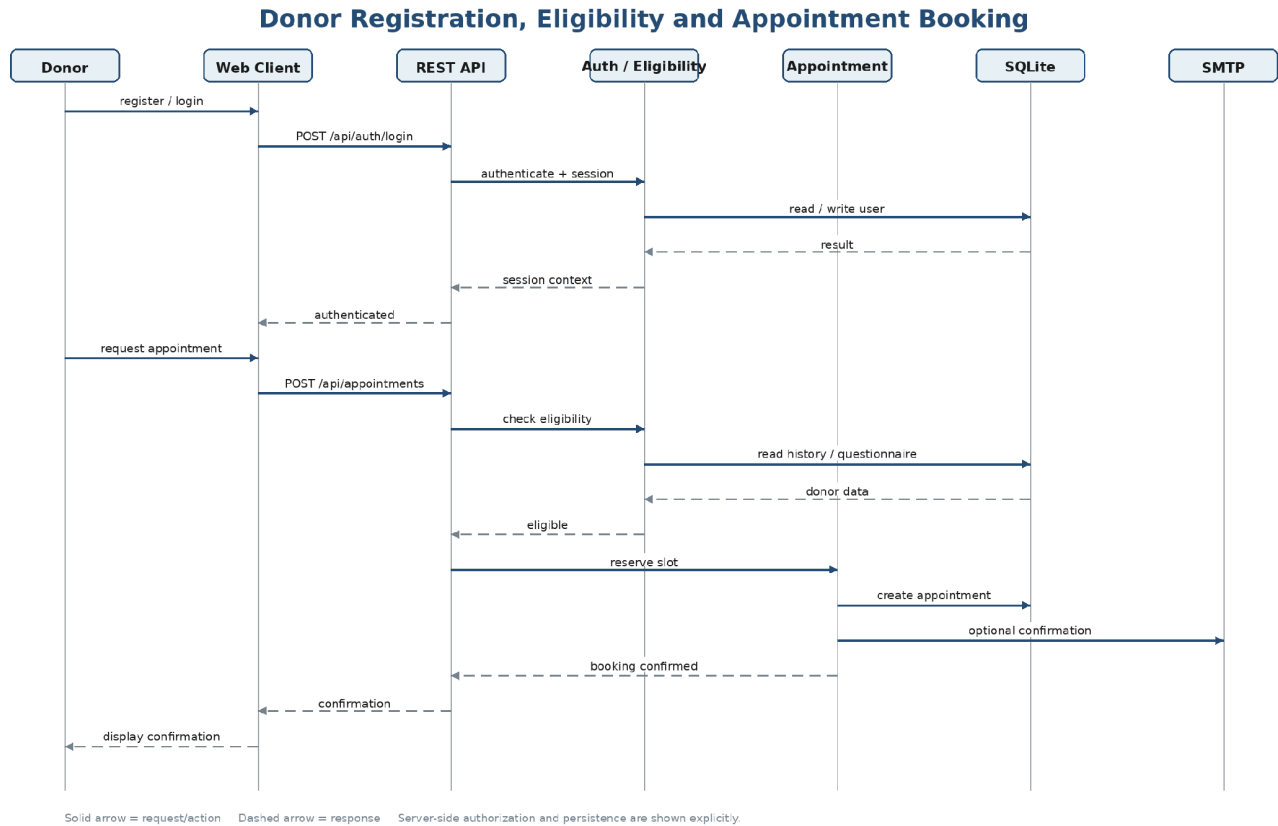


Figure 4.1 — Donor Registration, Eligibility, and Appointment Booking Sequence

The sequence enforces the SRS ordering rule that eligibility screening must occur before appointment confirmation. Authentication establishes the donor session first; the eligibility service checks configured conditions and donation interval; the appointment service then persists the slot reservation and optional notification.

4.4 Sequence Diagram — Hospital Requisition and Dispatch

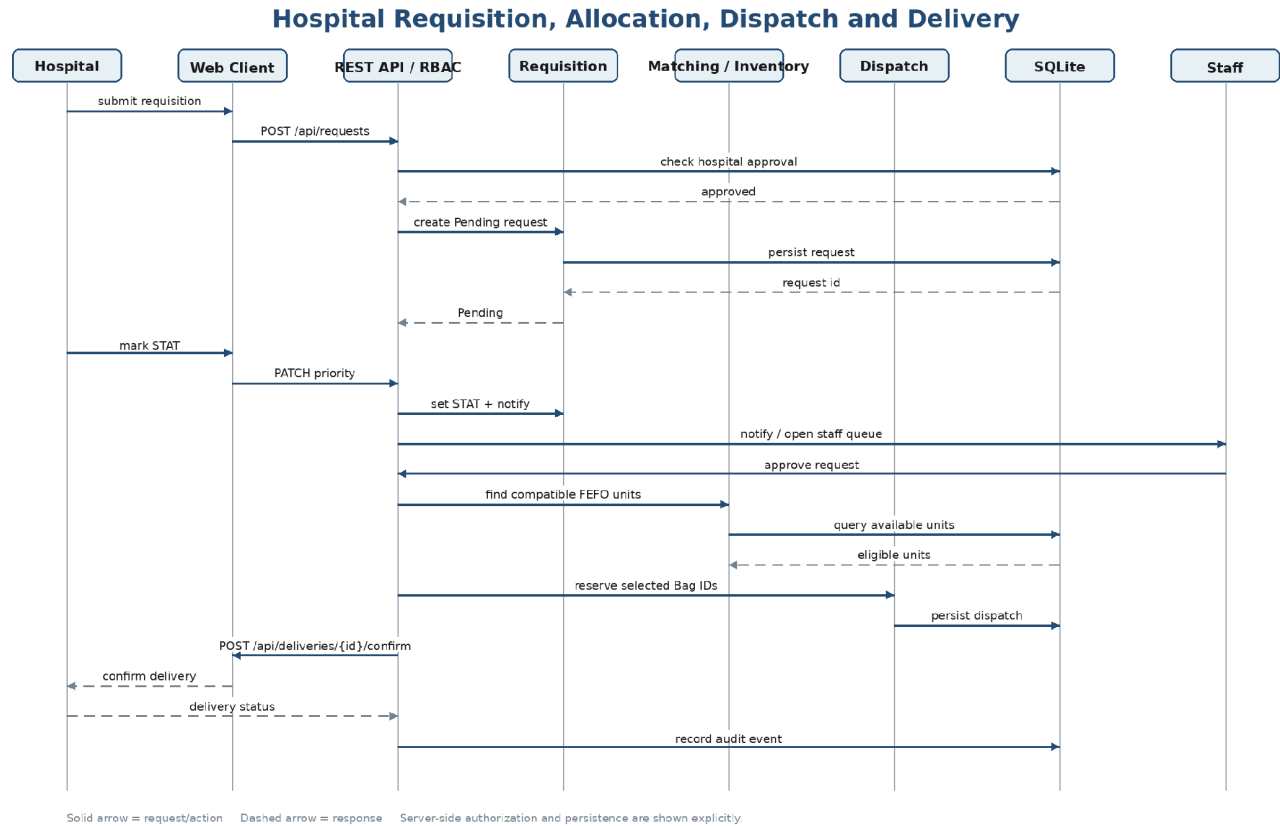


Figure 4.2 — Hospital Requisition, Allocation, Dispatch, and Delivery Confirmation Sequence

The flow shows server-side hospital approval checking, Pending request creation, STAT prioritization, staff approval, compatibility/FEFO selection, reservation, dispatch recording, and delivery confirmation. Each committed state transition is persistently recorded and is intended to emit an audit event.

4.5 API Design

The SRS requires a RESTful JSON API over HTTPS. The endpoint table below is the design contract for the major workflows; implementation may use Django REST Framework, Django views, Express routes, or equivalent framework mechanisms.

Method	Endpoint	Role	Purpose	SRS
POST	/api/auth/register	Public	Create donor account	F-001–F-003
POST	/api/auth/login	Public	Authenticate user / create session	F-004; SR-001, SR-006, SR-007
POST	/api/auth/password-reset/request	Public	Request password reset	F-005
POST	/api/auth/logout	Authenticated	Invalidate current session	SR-006
GET/PUT	/api/donors/me	Donor	View/update donor profile	F-006
GET	/api/donors/me/donations	Donor	View donation history	F-007
GET	/api/donors/me/eligibility	Donor	Return next eligible date / status	F-008–F-011
POST	/api/appointments	Donor	Book eligible donor into slot	F-012–F-016
PATCH	/api/appointments/{id}	Donor	Reschedule/cancel appointment	F-014–F-015
POST	/api/collections	Staff	Record collection and create Bag ID	F-017–F-021
POST	/api/blood-units/{bagId}/serology	Staff	Record required test results	F-024–F-027
GET	/api/inventory	Staff	Filter inventory / availability	F-028, F-033–F-039
POST	/api/blood-units/{bagId}/components	Staff	Record component separation	F-031–F-032
POST	/api/hospitals/register	Public / Hospital	Submit hospital registration	F-040
POST	/api/requests	Hospital	Submit blood requisition	F-043–F-046
PATCH	/api/requests/{id}/priority	Hospital	Mark request Emergency/STAT	F-047–F-048

POST	/api/requests/{id}/cross-match	Staff	Compute compatibility / record result	F-049–F-051
POST	/api/requests/{id}/allocate	Staff	Reserve specific Bag IDs	F-052
POST	/api/dispatch	Staff	Create dispatch and manifest	F-053–F-055
POST	/api/deliveries/{id}/confirm	Hospital	Confirm receipt / custody	F-056–F-057
GET	/api/audit	Admin	Read-only audit log	F-058–F-059
GET	/api/reports/inventory.csv	Admin	Export inventory report	F-064
GET	/api/reports/donations.csv	Admin	Export donation activity report	F-065
GET	/api/reports/requisitions.csv	Admin	Export requisition fulfilment report	F-066
GET	/api/public/availability	Public	Read-only stock availability	F-067–F-068

4.6 API Authentication and Authorization

Request Stage	Behavior	Security Reference
1. Transport	Require HTTPS/TLS.	SR-008
2. Session	Read authenticated session from secure cookie; reject absent/expired session when route is protected.	SR-001, SR-006
3. Role check	Authorize endpoint against role-permission matrix.	SR-003
4. Input validation	Validate request schema and business constraints.	SR-005
5. Service call	Execute domain operation using trusted service context.	F-001–F-068
6. Audit	Record required security/data-change event.	F-058, SR-008

4.7 Error Handling

HTTP / Condition	Meaning	Response Structure / Action
400	Invalid or malformed input	Return field-level validation errors; do not execute domain operation.
401	Missing or invalid authentication	Return generic authentication failure; do not disclose credential details.
403	Authenticated but not authorized	Return forbidden response; no protected resource is exposed.
404	Resource not found	Return stable resource-not-found error; do not reveal unrelated identifiers.
409	Business conflict	Examples: duplicate email/Bag ID, full slot, incompatible status transition.
429	Rate limit exceeded	Return retry guidance and enforce login/request throttle where configured.
500	Unexpected application error	Log server-side with timestamp/stack trace; return generic client-safe message (NF-006).

Example JSON error envelope: `{"error":{"code":"VALIDATION_ERROR","message":"Request contains invalid fields","fields":{"email":"Invalid email format"}}}`. Error details returned to clients must not contain passwords, stack traces, database credentials, or other secrets.

4.8 Validation and State Management

The domain model uses explicit status guards. A collected blood unit starts as Quarantined. Only after all required serology results are recorded as Pass may it become Available; any Fail makes it Discarded and non-allocatable. Once a unit is allocated and dispatched it becomes Dispatched, and after hospital confirmation it becomes Delivered. Expired units become Expired and are excluded from availability and allocation. Requisition status follows Pending → Approved/Partially Fulfilled/Rejected and then Fulfilled or Partially Fulfilled as units are dispatched.

Object	State Transition	Guard / Condition
Blood Unit	Collected → Quarantined	Created at collection
Blood Unit	Quarantined → Available	All required serology results = Pass
Blood Unit	Quarantined/Available → Discarded	Any serology result = Fail / safe-disposal action
Blood Unit	Available → Expired	Expiry date passed
Blood Unit	Available → Dispatched	Valid allocation + dispatch
Blood Unit	Dispatched → Delivered	Hospital receipt confirmation
Requisition	New → Pending	Successful submission

Requisition	Pending → Approved / Partially Fulfilled / Rejected	Staff action
Requisition	Approved → Fulfilled / Partially Fulfilled	Dispatch quantity determines final status

4.9 Audit Logging Design

Audit logging is implemented as an append-oriented record model. Each event stores actor identity, action, timestamp, affected entity/record ID, and sufficient context to reconstruct the change. The application does not expose edit/delete operations for audit records. Administrative UI access is read-only. Security events and business events use the same audit pipeline so the Test Plan can validate both functional and security accountability.

5. Requirements Traceability

5.1 Architecture-to-SRS Traceability

Architecture Element	Functional Requirements	Non-Functional Requirements	Security Requirements
Web / API	F-001–F-005, F-006–F-019, F-040–F-048, F-060–F-068	NF-007–NF-009	SR-001, SR-003, SR-005, SR-006
Auth / RBAC	F-001–F-005, F-040–F-042, F-060–F-063	NF-005	SR-001–SR-003, SR-006–SR-007
Collection / Unit	F-017–F-023	NF-004–NF-006	SR-004, SR-008
Testing / Quarantine	F-024–F-032	NF-004, NF-005	SR-004, SR-008
Inventory / FEFO	F-033–F-039	NF-001–NF-003, NF-014–NF-015	SR-004, SR-008
Hospital / Requisition	F-040–F-048	NF-001–NF-003	SR-001, SR-003, SR-005, SR-008
Matching / Allocation	F-049–F-052	NF-001–NF-003	SR-004, SR-008
Dispatch / Delivery	F-053–F-057	NF-004–NF-006	SR-003, SR-008
Audit / Reporting	F-058–F-066	NF-006, NF-010–NF-011	SR-008
Public Availability	F-067–F-068	NF-001–NF-009	SR-005, SR-008

5.2 Design-to-Test-Plan Traceability

Design Element	Primary Test Cases
Authentication / session	TC-AUTH-01, TC-SEC-02, TC-SEC-06, TC-SEC-07
Donor eligibility / appointment	TC-DNR-01, TC-ELG-01, TC-APT-01
Blood collection / Bag ID	TC-COL-01, TC-BAG-01
Serology / quarantine / components	TC-SER-01, TC-QNT-01, TC-CMP-01
Inventory / FEFO / expiry	TC-INV-01, TC-EXP-01
Hospital requisition / STAT	TC-AUTH-03, TC-REQ-01, TC-REQ-03
Compatibility / allocation	TC-MATCH-01
Dispatch / delivery	TC-DISP-01, TC-DEL-01
Audit / reports / public portal	TC-AUD-01, TC-RPT-01, TC-PUB-01
Performance / reliability / accessibility / portability	TC-NF-01, TC-NF-02, TC-NF-03, TC-NF-04, TC-NF-05, TC-NF-06
Security architecture	TC-SEC-01 through TC-SEC-08

The companion Test Plan contains the detailed test execution records and statuses. Until execution, the status of those tests remains "Not Run".

6. Design Decisions, Risks and Limitations

6.1 Key Design Decisions

- Use a layered client-server architecture to isolate UI, authorization, domain services, and persistence.
- Treat all security controls as server-side responsibilities; client-side hiding is never sufficient authorization.
- Use explicit blood-unit and requisition state transitions to prevent unsafe allocation.
- Keep API contracts stable across the permitted Django or Node.js backend implementations.
- Use SQLite as specified for the mini-project while keeping a clean data-access boundary for future migration.

6.2 Risks and Mitigations

Risk	Impact	Mitigation
SQLite concurrent-write limits	Reduced throughput during load	Keep deployment within SRS scope and measure against NF-001–NF-003.
Incorrect clinical configuration	Unsafe business decision	Make eligibility disqualifiers, expiry rules, and compatibility data configurable and test-driven.
SMTP unavailable	Email tests blocked	Keep on-screen confirmations as primary UI behavior; mark dependent email tests accordingly.

Framework divergence (Django vs Node)	Inconsistent implementation	Maintain the logical component and API contracts independent of framework.
Incomplete audit coverage	Reduced accountability	Use a centralized audit service invoked from all data-changing/security paths.

6.3 Limitations

- The design does not cover direct integration with laboratory equipment or national/regional blood-bank registries because those are outside the SRS scope.
- SQLite and single-host deployment are appropriate to the academic mini-project but not a production multi-hospital deployment model.
- Clinical policy values such as eligibility conditions, component shelf lives, and compatibility rules are represented as system rules/configuration; the SRS does not claim regulatory certification.

7. Conclusion

This specification translates the BBMS SRS into a coherent layered architecture and detailed design. The component structure separates the donor, hospital, inventory, testing, matching, dispatch, audit, and security responsibilities; the sequence diagrams show the principal end-to-end workflows; the REST API design defines the system interaction boundary; and the security architecture addresses authentication, RBAC, input protection, session security, rate limiting, TLS, and auditability. The traceability sections align the architecture and design with the SRS requirement identifiers and the companion Test Plan test cases.